

hosni@linux ~ \$ cat deep-dive.txt

/etc/passwd

+

/etc/shadow

The full picture of Linux account identity, authentication and the misconfigurations that put both at risk.

// CONTENTS

- 01 The Big Picture
- 02 /etc/passwd: the identity card
- 03 /etc/shadow: the vault
- 04 Why the x? History and the login flow
- 05 Tools, PAM and NSS
- 06 Misconfigurations and vulnerabilities
- 07 The audit toolkit
- 08 Hardening checklist
- 09 Related files
- 10 Cheat sheet

01 The Big Picture

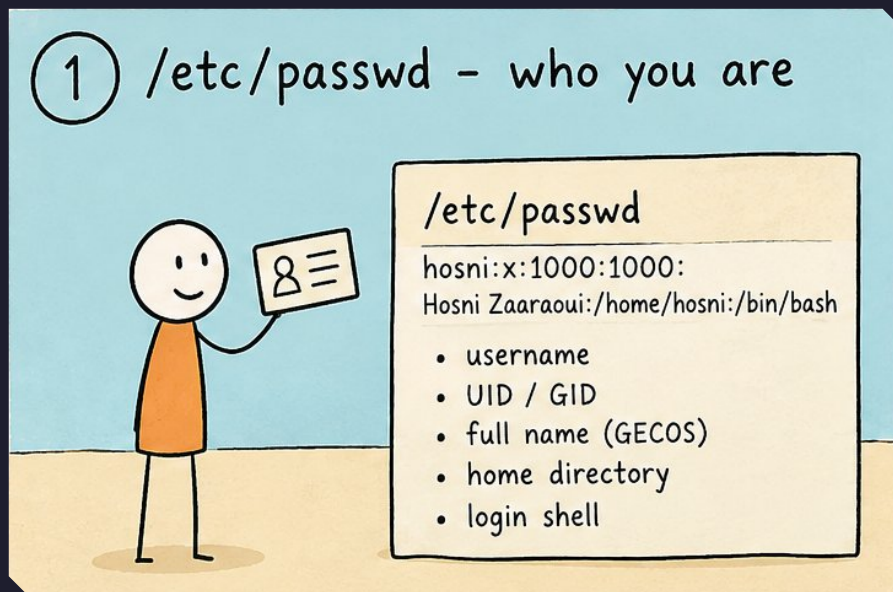
Two small text files sit at the heart of local Linux authentication. Most users know they exist, but every `:` separated field in them has a purpose, and small mistakes in either file can hand an attacker the whole system.

FILE	ANSWERS THE QUESTION	WHO CAN READ IT	HOLDS
<code>/etc/passwd</code>	Who is this user?	Everyone (world-readable)	Username, UID, GID, home, shell
<code>/etc/shadow</code>	How and when do they authenticate?	root (and the shadow group on Debian/Ubuntu)	Password hash, aging and expiry policy

REMEMBER `/etc/passwd` stores **who** the user is. `/etc/shadow` stores **how** and **when** they authenticate.

02 /etc/passwd: the identity card

Think of it as the user's identity card. Every line is one account, made of seven fields separated by colons.



• • • /etc/passwd

hosni:x:1000:1000:Hosni Zaaraoui:/home/hosni:/bin/bash

► Field by field

#	FIELD	VALUE	WHAT IT MEANS
1	Username	hosni	The account name used to log in.
2	Password placeholder	x	The real password hash lives in /etc/shadow .
3	UID	1000	User ID. The number Linux actually uses to identify the user internally.
4	GID	1000	Primary group ID of the user.
5	GECOS	Hosni Zaaraoui	User information, usually the full name.
6	Home directory	/home/hosni	The user's personal folder.
7	Login shell	/bin/bash	The program launched after login.

► Why these fields exist

- **Username** lets humans recognise the account.
- **UID** is what Linux really trusts for permissions. File ownership stores the number, not the name.
- **GID** connects the user to their primary group.
- **Home directory** tells applications where personal files live.
- **Shell** determines what starts after authentication.

► UID ranges worth knowing

UID	TYPICAL MEANING	NOTES
0	root	Full control. Any account with UID 0 is effectively root.
1 – 999	System and service accounts	Daemons such as www-data , sshd . Should not have an interactive shell.
1000+	Regular human users	Set by UID_MIN in /etc/login.defs .
65534	nobody	Unprivileged placeholder account for NFS and sandboxed services.

NOTE The GECOS field can hold comma-separated sub-fields (full name, room, phone). [chfn](#) edits them, which is another reason not to trust this field for anything security-relevant.

03 /etc/shadow: the vault

This file is readable only by privileged users because it contains password-related data. Each line has nine fields.



• • • /etc/shadow

hosni:\$y\$j9T\$abc123...:20359:0:99999:7:::

► Field by field

#	FIELD	EXAMPLE	WHAT IT MEANS
1	Username	hosni	Matches the account in <code>/etc/passwd</code> .
2	Password hash	\$y\$j9T\$...	The hashed password. Never the real password.
3	Last change	20359	Days since 1 Jan 1970 when the password last changed. (20359 is 2025-09-28.)
4	Minimum age	0	Days before the password may be changed again.
5	Maximum age	99999	Password lifetime in days. 99999 means it effectively never expires.

#	FIELD	EXAMPLE	WHAT IT MEANS
6	Warning period	7	Days before expiry when Linux starts warning the user.
7	Inactive period	(empty)	Days after expiry before the account is disabled.
8	Account expiry	(empty)	Day number (since epoch) when the account permanently expires.
9	Reserved	(empty)	Reserved for future use.

► More than just passwords

/etc/shadow also enforces security policy:

- Password expiration and mandatory password changes
- Advance warnings before expiry
- Automatic account disabling and account expiry

Reading the date fields

```
● ● ● bash
```

```
# convert a shadow day-number to a real date
```

```
> date -d "1970-01-01 +20359 days" +%F
2025-09-28
```

```
# what is today's day-number?
```

```
> echo $(( $(date +%s) / 86400 ))
```

► Anatomy of a hash

A modern hash is self-describing: `$id$parameters$salt$hash`. The prefix tells the system which algorithm to verify with.

PREFIX	ALGORITHM	STATUS
\$1\$	MD5-crypt	Broken. Replace it.
\$5\$	SHA-256-crypt	Acceptable, but dated
\$6\$	SHA-512-crypt	Long-time default, still common
\$2b\$ / \$2y\$	bcrypt	Strong, but less common in shadow files
\$y\$	yescrypt	Modern default (Debian 11+, Ubuntu 22.04+)

● ● ● dissecting a yescrypt hash

\$y\$j9T\$<salt>\$<hash>

\$y\$ algorithm id → yescrypt
 j9T cost params → work factor / memory settings
 <salt> random per-password value
 <hash> the actual derived key

NOTE Existing hashes keep their old algorithm until the user changes their password. Upgrading `ENCRYPT_METHOD` in `/etc/login.defs` does not rehash anything by itself.

► Special password values

Not every account has a usable password. The password field can also hold a marker instead of a hash.

VALUE	MEANING	CAVEAT
!hash	Password login locked (<code>passwd -l</code> prepends !)	The original hash is preserved, so unlocking restores it.
! or !!	Password never set / account locked	!! is the RHEL-style "never set"; Debian tends to use !.
*	Password authentication disabled	Common for system accounts. No password can ever match.
(empty)	No password at all	Can allow passwordless login if PAM permits <code>nullok</code> . Treat as critical.

WARNING Locked or disabled password does **not** always mean the account cannot log in. SSH keys, and other non-password methods, can still work. To truly disable an account also set an expiry (`chage -E 0 user`) and a non-login shell.

04 Why the x? History and the login flow

If the second field is always `x`, why keep it? For **backward compatibility**, and because it acts as a marker that tells Linux where the password is stored.

► The original design

Early Unix stored the hash right in `/etc/passwd`:

● ● ● the old way

```
hosni:$1$abc123...:1000:1000:Hosni:/home/hosni:/bin/bash
```

The problem: `/etc/passwd` **must** be world-readable because countless programs need to map UIDs to names. That meant anyone could copy every password hash and try to crack them offline. The fix was shadow passwords: move the hashes to a file only root can read.

► What the second field can hold

VALUE	MEANING
x	Look in <code>/etc/shadow</code> . The normal modern setup.
* or !	Password login disabled or account locked.
(empty)	On some systems, no password required. Usually insecure.
<hash>	A legacy layout with the hash stored directly. Should not exist on a modern system.

So `x` is not the password. It is a **pointer** that preserves the original file format while keeping hashes out of a world-readable file.

► What happens when you log in

1. Linux reads your username in `/etc/passwd`.
2. It sees `x` in the password field.
3. It fetches the real hash from `/etc/shadow`.
4. It hashes what you typed (using the same algorithm and salt) and compares the two.
5. It checks aging rules: expired password, expired account, inactive period.

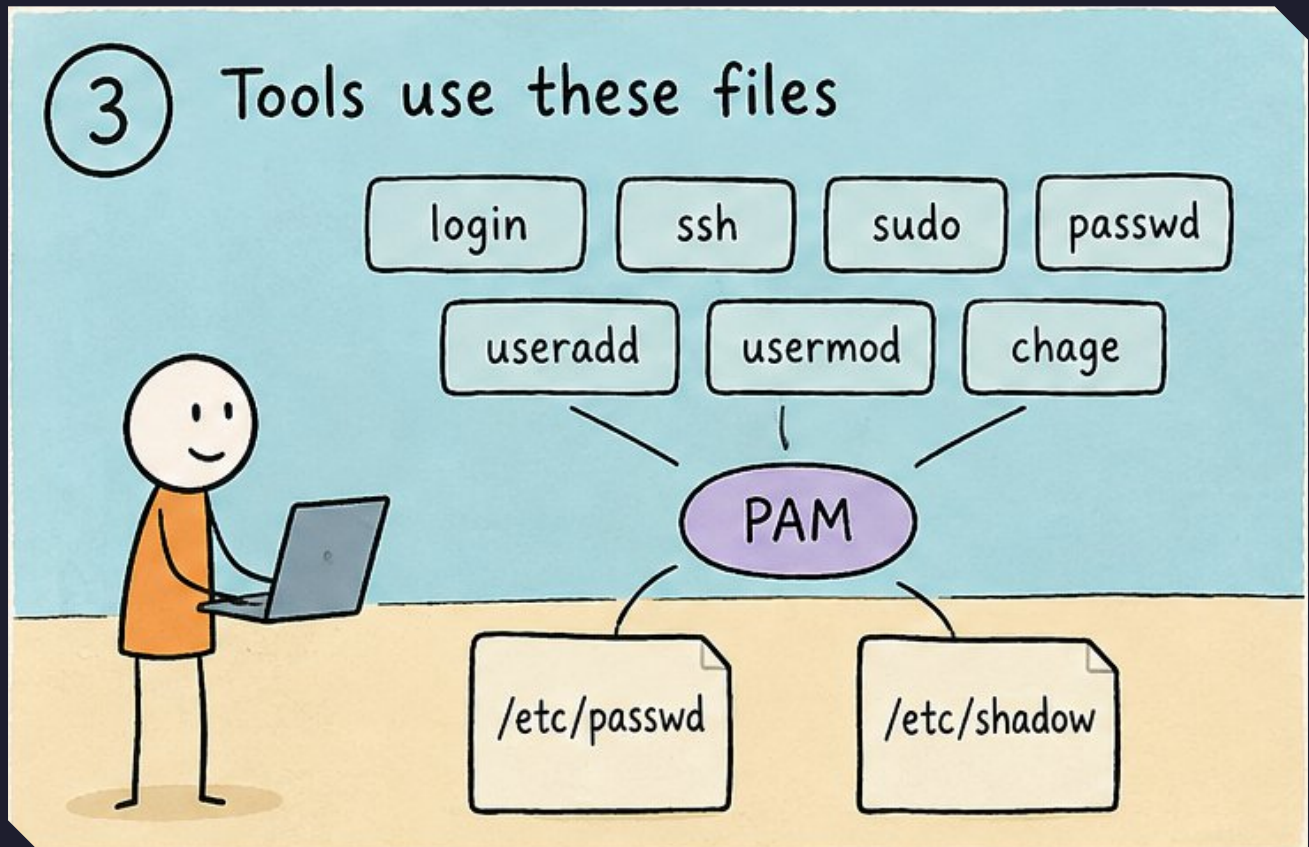
● ● ● both files together

```
/etc/passwd          /etc/shadow
-----
hosni:x:1000:1000:...  →  hosni:$y$j9T$...:20359:0:99999:7:::
```

TIP This split lets normal users read account information (needed by `ls -l`, `id`, `mail`, and more) without exposing password hashes.

05 Tools, PAM and NSS

Almost nothing edits these files by hand. Tools read and modify them indirectly, usually through PAM.



► Authentication tools

TOOL	USES IT FOR
<code>login</code>	Local console login
<code>sshd</code>	SSH authentication
<code>su</code>	Switching user
<code>sudo</code>	Validating user identity (through PAM)
<code>passwd</code>	Changing passwords and updating <code>/etc/shadow</code>

► User management tools

These safely modify the files instead of relying on manual edits.

COMMAND	PURPOSE
useradd	Creates a user
usermod	Modifies user properties
userdel	Removes a user
passwd	Sets or locks passwords
chage	Manages password aging
chsh	Changes the login shell
chfn	Updates GECOS information
pwck	Verifies the integrity of passwd and shadow
pwconv / pwunconv	Convert to or from shadow passwords
vipw / vipw -s	Lock-then-edit /etc/passwd or /etc/shadow safely

► Information lookup tools

```
● ● ● bash
```

```
> id hosni
> whoami
> getent passwd hosni
> sudo getent shadow hosni  # root only
> finger hosni             # if installed
```

TIP Prefer [getent](#) over reading the files directly. Modern systems can source users from LDAP, Active Directory, SSSD or systemd, not only [/etc/passwd](#).

► PAM: the hidden layer

PAM (Pluggable Authentication Modules) is what most login programs actually talk to. It checks the username, retrieves the hash from [/etc/shadow](#), verifies the password and applies policies such as complexity, lockout and aging.

```
● ● ● flow
```

```
SSH / login / su / sudo
|
```

```
graph TD; PAM --> pam_d["/etc/pam.d/*"]; pam_d --- passwd["/etc/passwd"]; pam_d --- shadow["/etc/shadow"];
```

► NSS: where do users come from?

The Name Service Switch decides which sources answer user and password lookups, and in which order.

```
/etc/nsswitch.conf
```

```
passwd:  files systemd sss
group:   files systemd sss
shadow:  files sss
```

`files` means the local `/etc` files. Extra sources like `sss` (SSSD) or `ldap` mean an account may exist without appearing in `/etc/passwd`, so audits based only on `grep` can miss it.

► Why admins rarely edit manually

Although they are plain text, `useradd`, `passwd` and `usermod` also validate the format, prevent duplicate UIDs, lock the files during changes and keep both files in sync. When manual editing is needed, use `vipw` and `vipw -s`, which lock the files first to prevent corruption from concurrent changes.

06 Misconfigurations and vulnerabilities

Most account-file problems come down to three questions: can the wrong people **read** it, can the wrong people **write** it, and does an account exist that should not authenticate the way it does.



#	ISSUE	WHY IT MATTERS	SEVERITY
1	World-readable /etc/shadow	Hashes can be stolen for offline cracking.	CRITICAL
2	World-writable /etc/passwd or shadow	Anyone can create or alter accounts, including root-equivalents.	CRITICAL
3	Extra UID 0 account	A second, possibly overlooked, root.	CRITICAL
4	Empty password field	Possible passwordless login.	CRITICAL
5	Interactive shell on service accounts	Compromised service becomes an interactive foothold.	HIGH
6	Weak or legacy hashes (\$1\$, \$5\$)	Faster to crack if the file leaks.	HIGH
7	Weak password aging (99999)	Passwords never rotate or expire.	MEDIUM
8	Locked vs disabled confusion	Accounts stay reachable via SSH keys.	MEDIUM

#	ISSUE	WHY IT MATTERS	SEVERITY
9	Readable backups (passwd-, shadow-)	Same exposure as the original file.	HIGH
10	Stale or orphaned accounts	Unused accounts are unmonitored entry points.	MEDIUM

► 1. World-readable /etc/shadow

Expected permissions vary by distribution:

DISTRO FAMILY	EXPECTED MODE	OWNER:GROUP
Debian, Ubuntu	-rw-r----- (640)	root:shadow
RHEL, Fedora, Rocky, Alma	----- (000)	root:root
Strict / hardened baselines	-rw----- (600)	root:root

The dangerous case is any **other** bit being set, or membership in the **shadow** group being handed out casually.

● ● ● check and fix

```
> ls -l /etc/shadow
-rw-r----- 1 root shadow 1234 Sep 28 10:00 /etc/shadow
```

fix on a Debian-style system

```
> sudo chown root:shadow /etc/shadow
> sudo chmod 640 /etc/shadow
```

fix on a strict / RHEL-style system

```
> sudo chown root:root /etc/shadow
> sudo chmod 600 /etc/shadow
```

CRITICAL If everyone can read **/etc/shadow**, attackers can copy the hashes and brute-force them offline at their leisure, with no lockouts and no logs.

► 2. Incorrect /etc/passwd permissions

Expected: **-rw-r--r--** (644, owned by root). Readable by everyone, writable only by root. Something like **-rw-rw-rw-** means anyone can edit accounts, for example by adding a new UID 0 user.

▶ 3. Empty password field

● ● ● dangerous

```
hosni::1000:1000:...
```

Depending on PAM configuration ([nullok](#)), this may allow passwordless logins. Check both files, since an empty second field in [/etc/shadow](#) is just as bad.

▶ 4. UID 0 on another account

● ● ● dangerous

```
backup:x:0:0:Backup:/home/backup:/bin/bash
```

Any account with UID 0 is effectively another root account. The expected output of the check below is [root](#) only.

● ● ● bash

```
> awk -F: '$3==0 {print $1}' /etc/passwd
```

▶ 5. Interactive shell for service accounts

● ● ● dangerous vs better

```
www-data:x:33:33::/var/www:/bin/bash
```

```
www-data:x:33:33::/var/www:/usr/sbin/nologin
```

Service accounts should use [/usr/sbin/nologin](#) or [/bin/false](#).

● ● ● bash

```
> grep -vE '(nologin|false)$' /etc/passwd
```

▶ 6. Weak password aging

A shadow line ending in [:0:99999:7:](#) means passwords effectively never expire. Inspect a user with [chage -l](#).

● ● ● bash

```
> sudo chage -l hosni
```

```
> sudo chage -M 90 -m 1 -W 7 hosni # example policy
```

NOTE `PASS_MAX_DAYS` and friends in `/etc/login.defs` only apply to **newly created** accounts. Existing users need `chage`.

► 7. Locked vs disabled accounts

The symbols look similar but behave differently. Misunderstanding them can leave accounts reachable through SSH keys while password login is blocked. Use the table in section 03 as reference, and combine `passwd -l` with `chage -E 0` and a `nologin` shell when you need a truly dead account.

► 8. Forgotten extras

- **Backup files** `/etc/passwd-` and `/etc/shadow-` are copies made by the tools. Their permissions must be just as strict as the originals.
- **Duplicate UIDs** make two accounts share ownership of files.
- **Missing home directories or dead shells** hint at deleted or hand-edited accounts.
- **Password aging is not key aging.** An expired password does not stop SSH key logins.

07 The audit toolkit

A single copy-paste-friendly reference. Run these with appropriate privileges on systems you are authorised to audit.

► Identity checks

● ● ● passwd audit

accounts with UID 0 (expect: root only)

➤ `awk -F: '$3==0 {print $1}' /etc/passwd`

accounts that have a login shell

➤ `grep -E '/bin/(bash|sh|zsh)$' /etc/passwd`

accounts whose shell is NOT nologin/false

➤ `grep -vE '(nologin|false)$' /etc/passwd`

duplicate UIDs and duplicate usernames

➤ `cut -d: -f3 /etc/passwd | sort -n | uniq -d`

➤ `cut -d: -f1 /etc/passwd | sort | uniq -d`

```
# empty password field in passwd
> awk -F: '$2=="' {print $1}' /etc/passwd
```

► Authentication checks

● ● ● shadow audit

```
# empty password hashes (critical)
> sudo awk -F: '($2=="') {print $1}' /etc/shadow

# accounts that actually have a usable hash
> sudo awk -F: '$2 !~ /^[!*/' {print $1}' /etc/shadow

# survey of hash algorithms in use
> sudo awk -F: '$2 ~ /^[^\$/' {split($2,a,"$"); print a[2]} /etc/shadow \
| sort | uniq -c

# password aging for one user
> sudo chage -l username

# status of every account (P=usable, L=locked, NP=no password)
> sudo passwd -Sa
```

► File integrity and permissions

● ● ● integrity

```
> ls -l /etc/passwd /etc/shadow /etc/passwd- /etc/shadow-
> ls -l /etc/group /etc/gshadow

# consistency check (read-only mode with -r)
> sudo pwck -r
> sudo grpck -r
```

► Activity and logout

● ● ● logins

```
> last -n 20          # recent logins
> lastlog | grep -v Never # accounts that have logged in
> sudo faillock --user hosni # failed-attempt counter (if pam_faillock)
```

08 Hardening checklist

► Permissions and integrity

- `/etc/passwd` is 644 root-owned. `/etc/shadow` is 640/600/000 per distro policy.
- Backups `/etc/passwd-` and `/etc/shadow-` have equally strict permissions.
- Only `root` has UID 0. No duplicate UIDs or usernames.
- Run `pwck` and `grpck` after any manual change. Use `vipw` for manual edits.

► Passwords and hashes

- No empty password fields in either file.
- Set `ENCRYPT_METHOD YESCRYPT` (or SHA512 on older systems) in `/etc/login.defs`.
- Force rehash of old accounts by requiring a password change (`chage -d 0 user`).
- Enforce complexity with `pam_pwquality`, and lockout with `pam_faillock`.

► Accounts

- Service accounts use `/usr/sbin/nologin` or `/bin/false`.
- Set sane aging defaults (`PASS_MAX_DAYS`, `PASS_MIN_DAYS`, `PASS_WARN_AGE`) and apply them to existing users with `chage`.
- Expire and remove stale accounts. Use `chage -E` for time-limited access.
- Prefer SSH keys with passphrases and consider disabling password SSH login.
- Lock direct root password login and use `sudo` where your policy allows.

► Key settings in `/etc/login.defs`

• • • `/etc/login.defs`

```
ENCRYPT_METHOD    YESCRYPT
PASS_MAX_DAYS    90
PASS_MIN_DAYS    1
PASS_WARN_AGE    7
UID_MIN          1000
UMASK            027
```

09 Related files

These files work alongside [passwd](#) and [shadow](#). A full account audit should look at them too.

FILE	ROLE
/etc/group	Group names, GIDs and membership. The group equivalent of passwd .
/etc/gshadow	Group passwords and group administrators. The group equivalent of shadow .
/etc/passwd - /etc/shadow -	Backup copies written by account tools. Keep permissions strict.
/etc/login.defs	Defaults for new accounts: UID ranges, hash method, aging.
/etc/nsswitch.conf	Which sources (files, LDAP, SSSD, systemd) answer user lookups.
/etc/pam.d/	PAM stacks per service: complexity, lockout, nullok behaviour.
/etc/shells	Valid login shells. Programs like chsh consult it.
/etc/sudoers	Who may run what as whom. Edit only with visudo .
~/.ssh/authorized_keys	Key-based login that bypasses shadow password state.

10 Cheat sheet

► Two-line summary

REMEMBER `/etc/passwd` = **who** the user is. `/etc/shadow` = **how** and **when** they authenticate. `x` is the pointer between them.

► Field maps

• • • /etc/passwd (7 fields)

name:**x**:**UID**:**GID**:**GECOS**:**home**:**shell**

• • • /etc/shadow (9 fields)

name:**hash**:**lastchg**:**min**:**max**:**warn**:**inactive**:**expire**:**reserved**

► Quick lookups

I WANT TO...	COMMAND
See a user's identity	<code>id user</code> <code>getent passwd user</code>
See password aging	<code>sudo chage -l user</code>
Force a password change at next login	<code>sudo chage -d 0 user</code>
Lock / unlock a password	<code>sudo passwd -l user</code> <code>passwd -u user</code>
Expire an account now	<code>sudo chage -E 0 user</code>
Find extra root accounts	<code>awk -F: '\$3==0' /etc/passwd</code>
Check every account's state	<code>sudo passwd -Sa</code>
Edit safely	<code>sudo vipw</code> <code>sudo vipw -s</code>
Verify file consistency	<code>sudo pwck -r && sudo grpck -r</code>

► Hash prefixes at a glance

\$1\$	\$5\$	\$6\$	\$Y\$
MD5	SHA-256	SHA-512	yescrypt

